Reconciling High-Level Optimizations and Low-Level Code in LLVM

JUNEYOUNG LEE, Seoul National University, Korea
CHUNG-KIL HUR, Seoul National University, Korea
RALF JUNG, MPI-SWS, Germany
ZHENGYANG LIU, University of Utah, USA
JOHN REGEHR, University of Utah, USA
NUNO P. LOPES, Microsoft Research, UK

LLVM miscompiles certain programs in C, C++, and Rust that use low-level language features such as raw pointers in Rust or conversion between integers and pointers in C or C++. The problem is that it is difficult for the compiler to implement aggressive, high-level memory optimizations while also respecting the guarantees made by the programming languages to low-level programs. A deeper problem is that the memory model for LLVM's intermediate representation (IR) is informal and the semantics of corner cases are not always clear to all compiler developers.

We developed a novel memory model for LLVM IR and formalized it. The new model requires a handful of problematic IR-level optimizations to be removed, but it also supports the addition of new optimizations that were not previously legal. We have implemented the new model and shown that it fixes known memory-model-related miscompilations without impacting the quality of generated code.

CCS Concepts: • **Software and its engineering → Semantics**; **Compilers**;

Additional Key Words and Phrases: IR Memory Model, LLVM

ACM Reference Format:
Juneyoung Lee, Chung-Kil Hur, Ralf Jung, Zhengyang Liu, John Regehr, and Nuno P. Lopes. 2018. Reconciling High-Level Optimizations and Low-Level Code in LLVM. *Proc. ACM Program. Lang.* 2, OOPSLA, Article 125 (November 2018), 28 pages. https://doi.org/10.1145/3276495

1 INTRODUCTION

The *memory model* for a programming language determines how programs are permitted to observe and modify storage. For example, references in Java are pointer-like in that they uniquely identify objects in memory, but programs are not allowed to construct references into the middle of objects or to fabricate references from scratch. In contrast, a truly low-level programming language, such as assembly language, allows arbitrary memory locations to be inspected and modified with no restrictions whatsoever on how addresses are computed.

C and C++ occupy an interesting niche. They are intended to be low-level languages; systems software—operating system kernels, virtual machine managers, embedded firmware, programming language runtimes, etc.—tends to be built in one or the other. To support these applications, pointers

Authors' addresses: Juneyoung Lee, Seoul National University, Korea; Chung-Kil Hur, Seoul National University, Korea; Ralf Jung, MPI-SWS, Germany; Zhengyang Liu, University of Utah, USA; John Regehr, University of Utah, USA; Nuno P. Lopes, Microsoft Research, UK.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2475-1421/2018/11-ART125
https://doi.org/10.1145/3276495